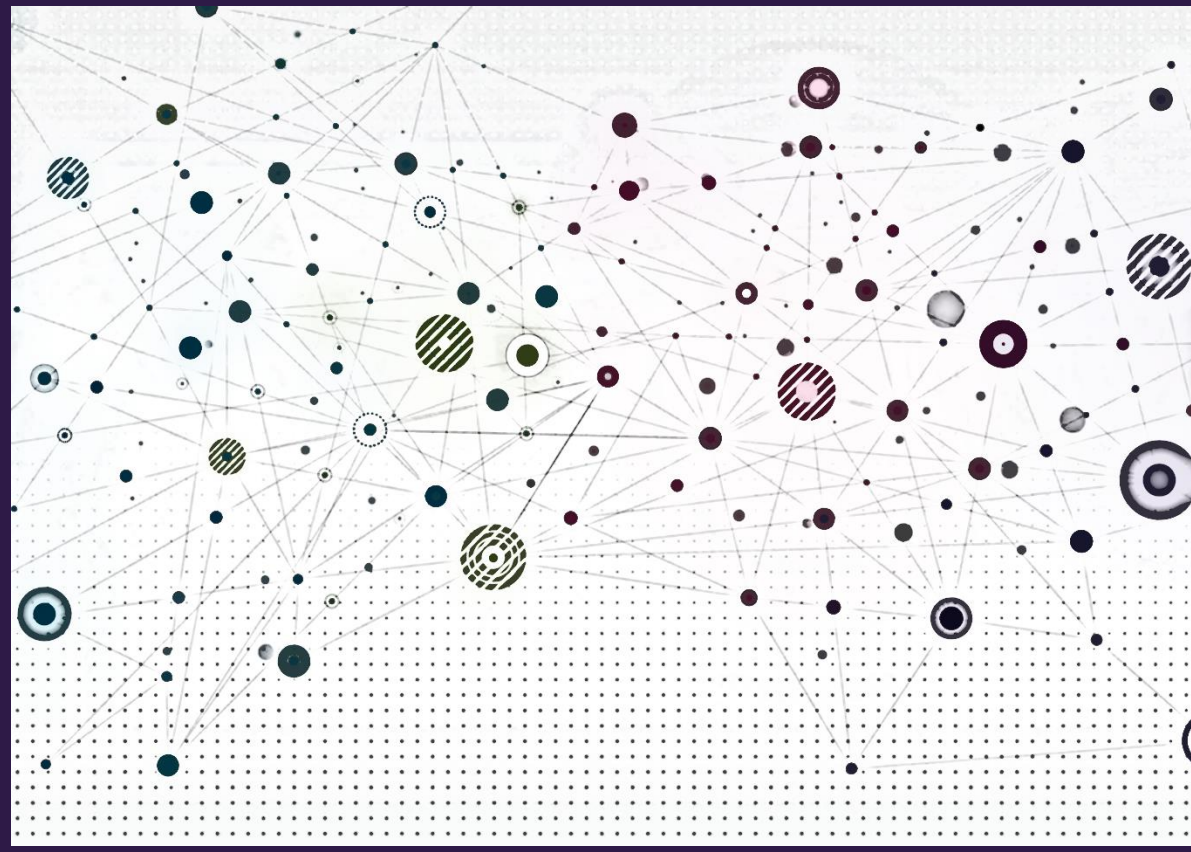
School of COMPUTING
Clemson University

Relational Deep Learning



Overview

- Relational databases
- Relational deep learning

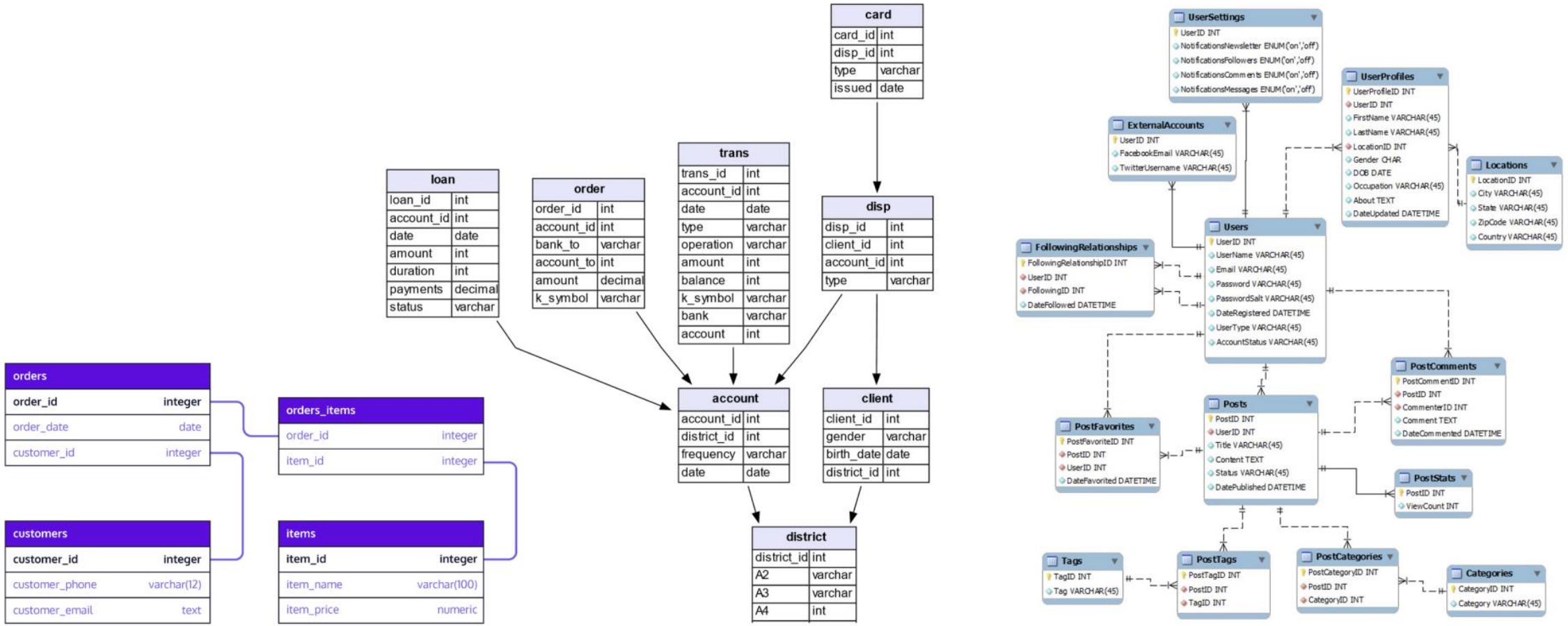
School of COMPUTING Clemson University

Relational databases

Relational databases

All technology stacks rely on a database management system (DBMS)

Most (not all) use relational databases



Relational databases

- Data is stored in a set of tables
- Each table stores entities in rows that:
  - Have the same features
  - Have a unique primary key (PK)
  - May have one or more foreign keys (FK) connecting the entity to another table
  - May have a time stamp

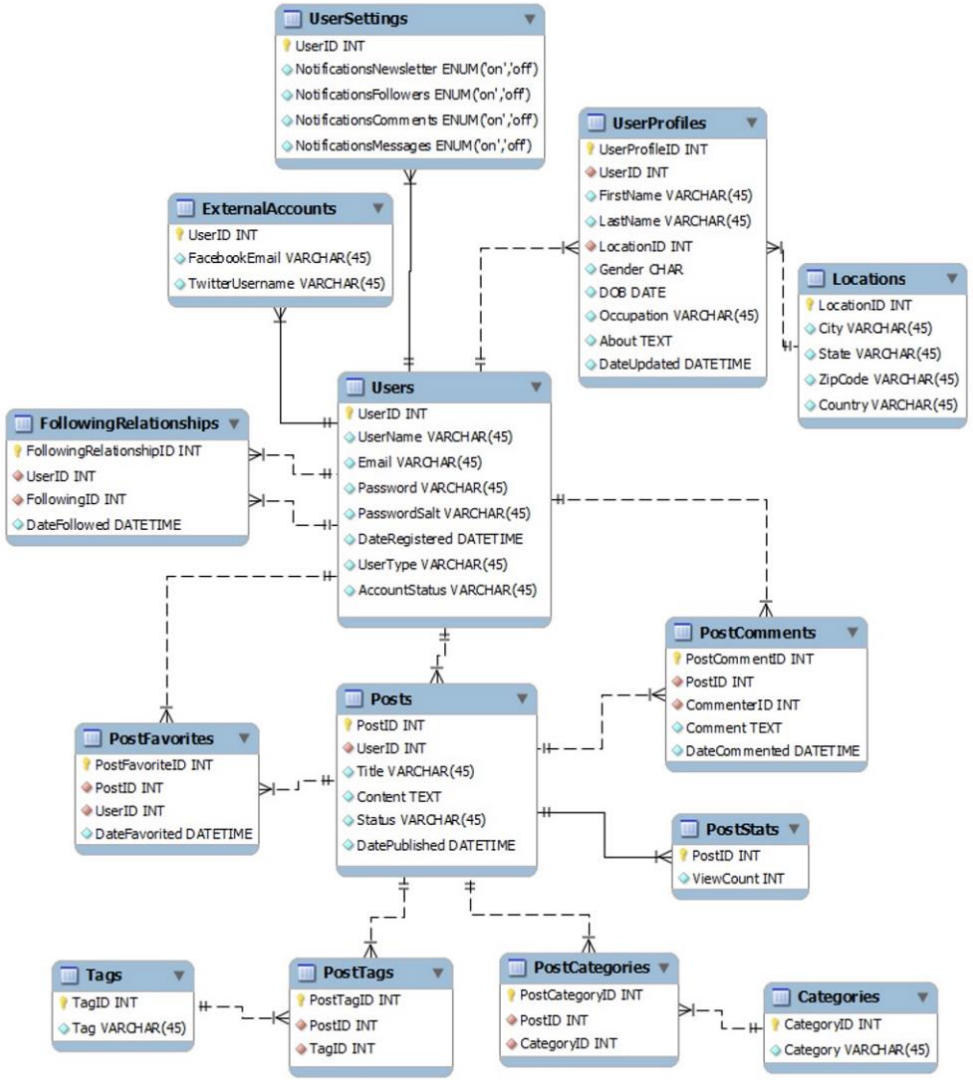
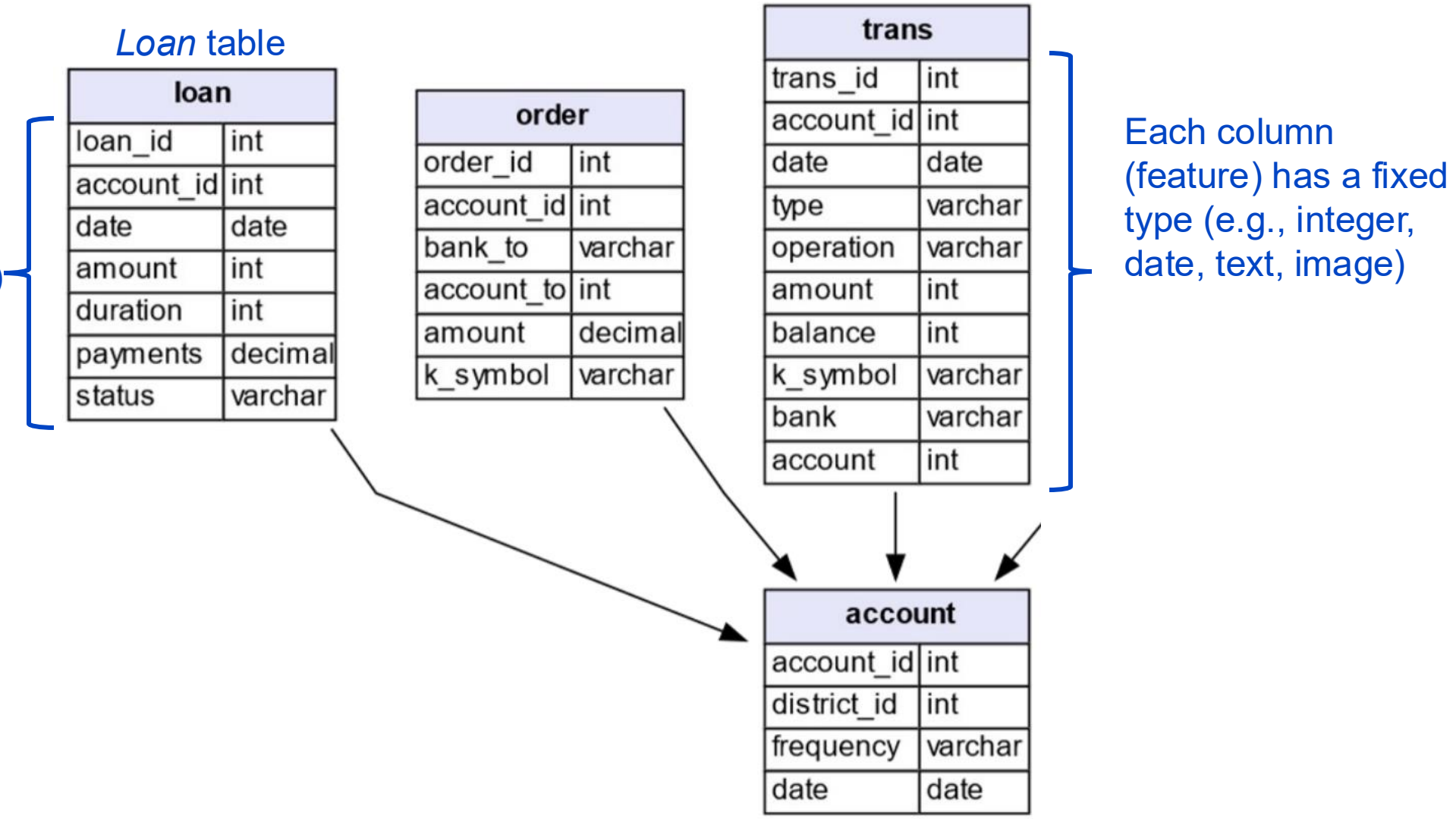


Image credit: Jure Leskovec, http://cs224w.stanford.edu

Relational database schema

Loan table

Each table has a distinct set of columns (features)



Each column (feature) has a fixed type (e.g., integer, date, text, image)

Image credit: Jure Leskovec, http://cs224w.stanford.edu

Relational database schema

Many tables include a timestamp feature indicating when the entity was added to the database

Loan table

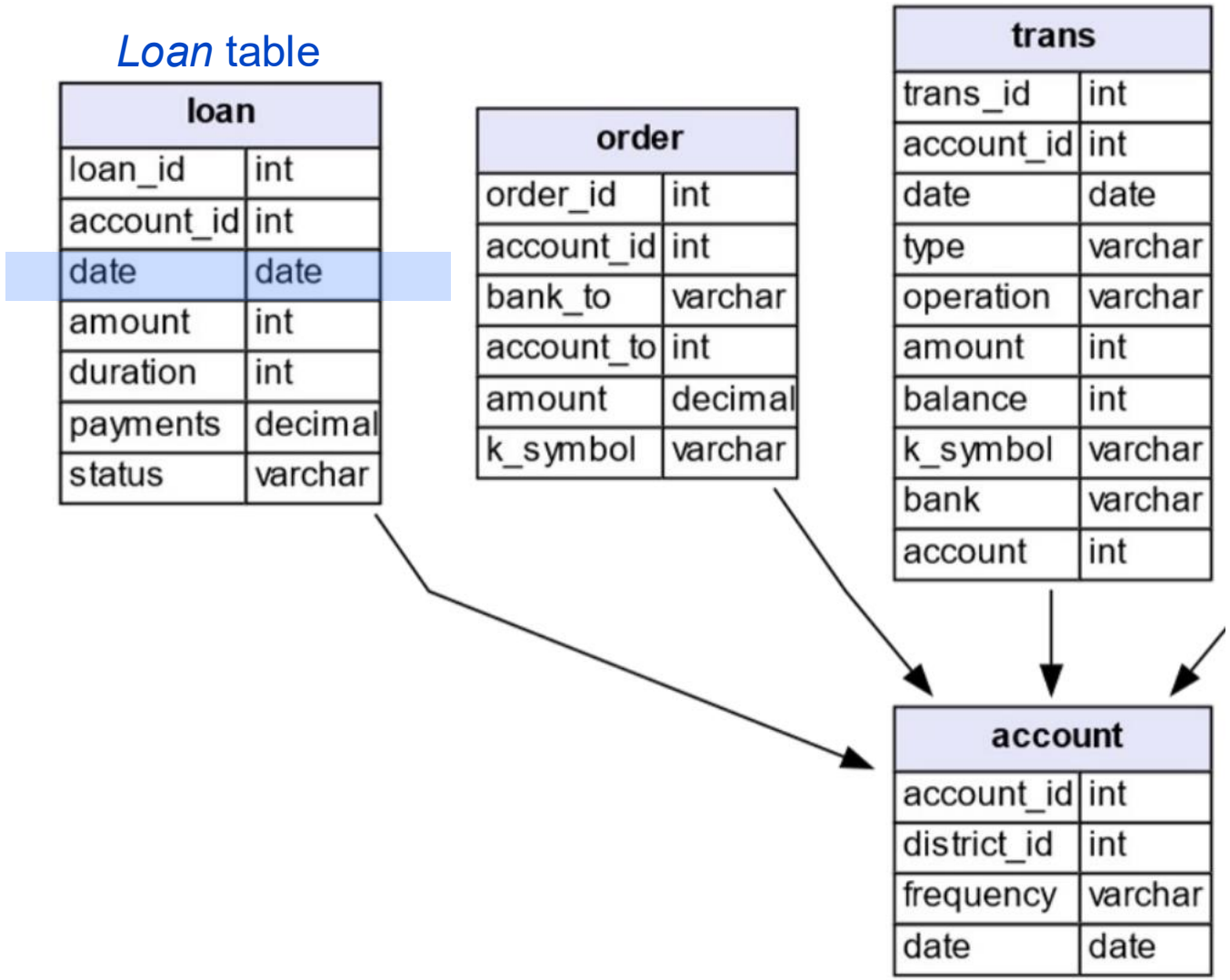


Image credit: Jure Leskovec, http://cs224w.stanford.edu

Relational database schema

Primary Key: loan_id distinct for every row in the table

Foreign Key: account_id links a row to a primary key value (row) in another table

Loan table

loan	
loan_id	int
account_id	int
date	date
amount	int
duration	int
payments	decimal
status	varchar

order	
order_id	int
account_id	int
bank_to	varchar
account_to	int
amount	decimal
k_symbol	varchar

trans	
trans_id	int
account_id	int
date	date
type	varchar
operation	varchar
amount	int
balance	int
k_symbol	varchar
bank	varchar
account	int

account	
account_id	int
district_id	int
frequency	varchar
date	date

Account table

Primary Key: account_id

Primary Key: different tables may use the same name for the primary key. They are distinct by the combination of table name (which cannot be duplicated) and primary key name

Image credit: Jure Leskovec, http://cs224w.stanford.edu

Relational database schema

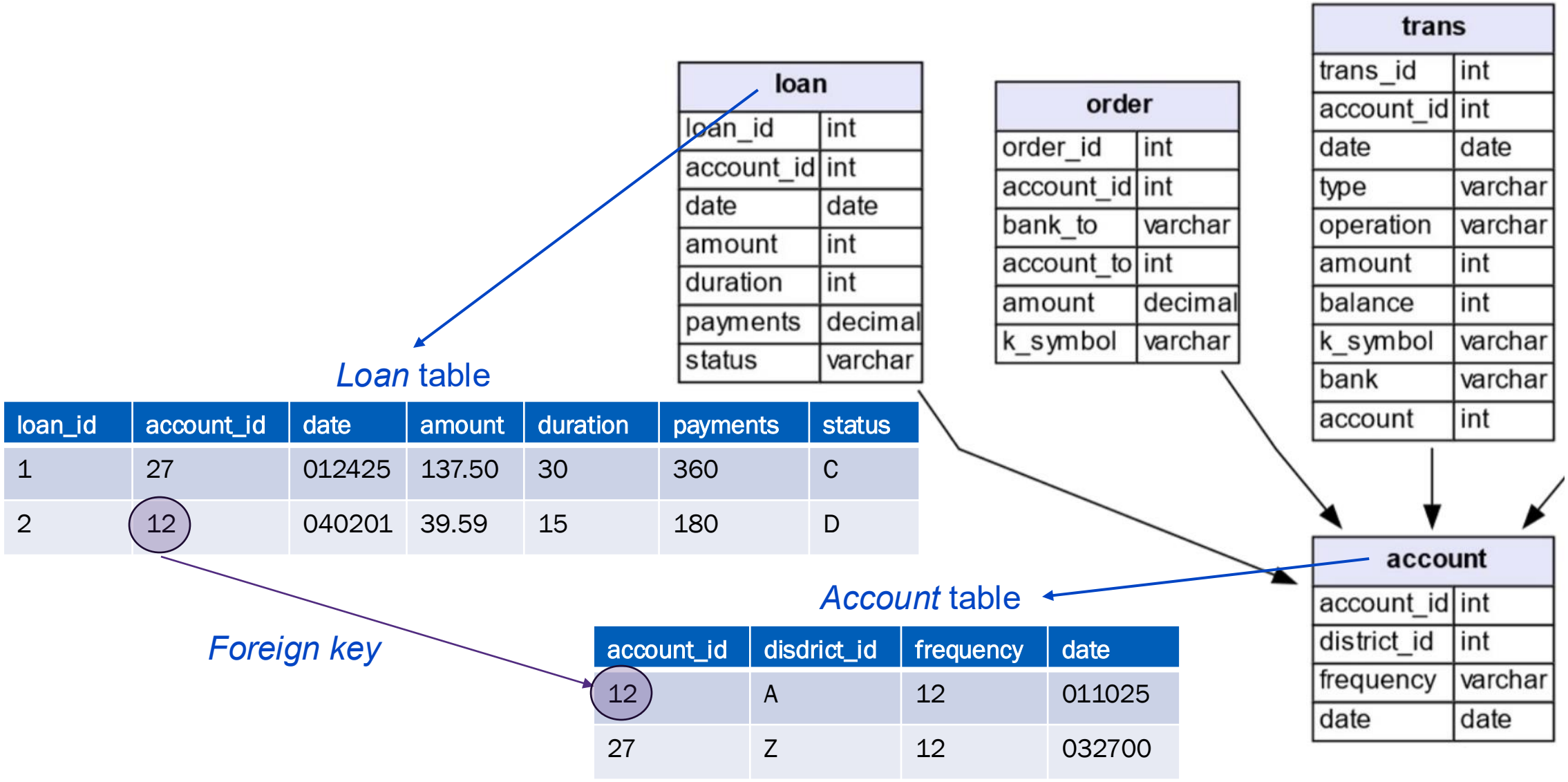


Image credit: Jure Leskovec, http://cs224w.stanford.edu

Predictions with relational databases

- Which products will a user purchase in the next 7 days?
- Will an active user churn in the next 90 days?
- What will be the total sales for each product in the next 30 days?

Image credit: Jure Leskovec, http://cs224w.stanford.edu

Predictions with relational databases

- Will a patient return if discharged from the hospital?
- Which hospital admissions have greatest risk to life in the next 24 hours

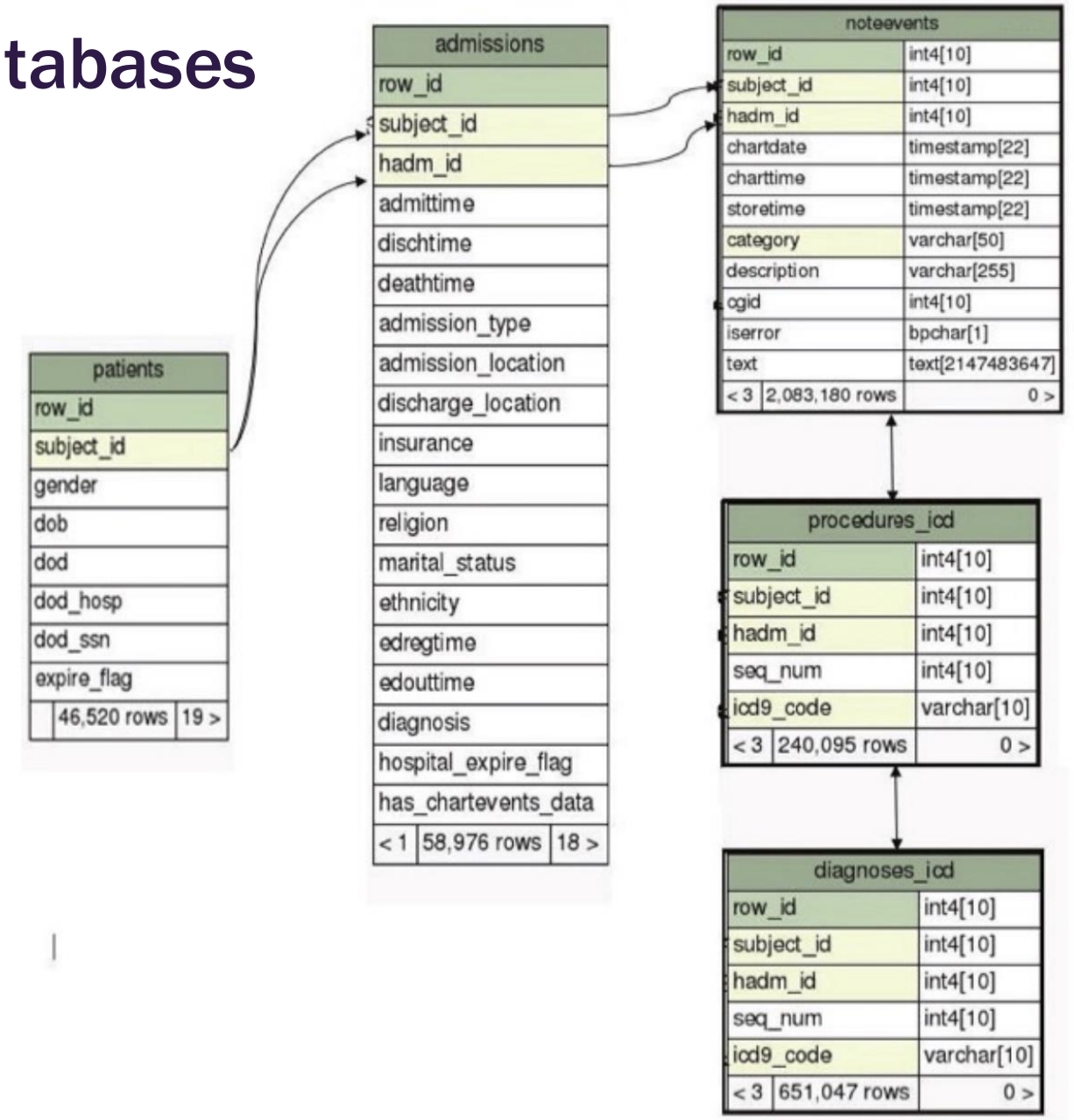


Image credit: Jure Leskovec, http://cs224w.stanford.edu

Standard approach to AI models on relational databases

- Feature selection / engineering using subset of tables / features
- Only a limited set of data is used
- High potential for point-in-time errors / information leakage

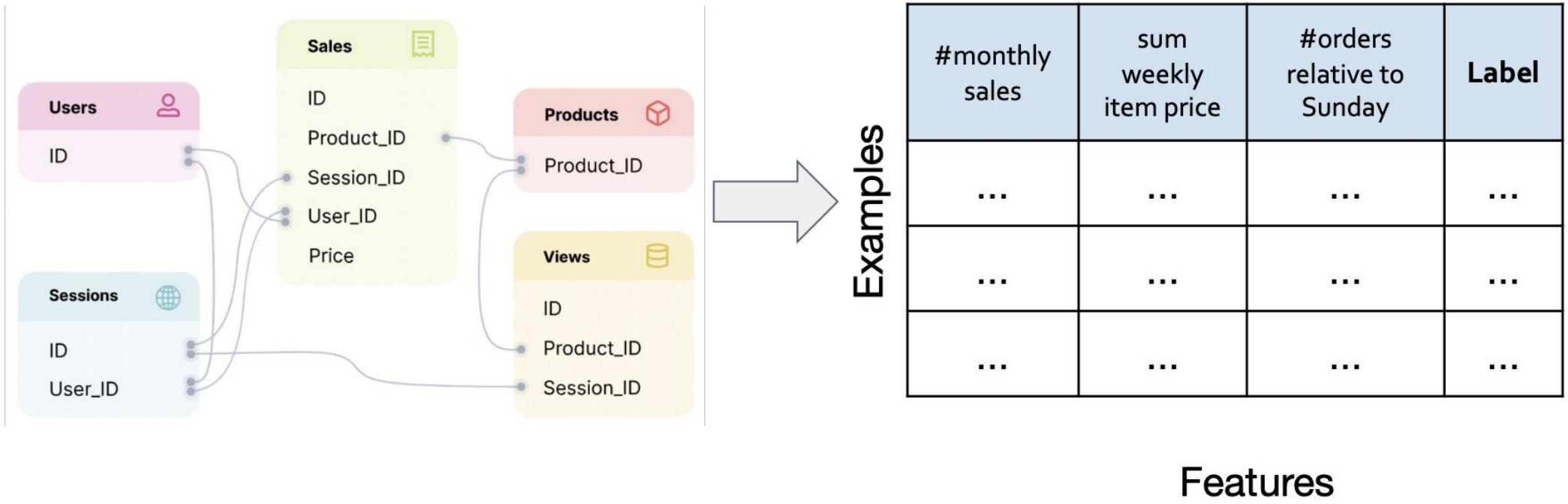
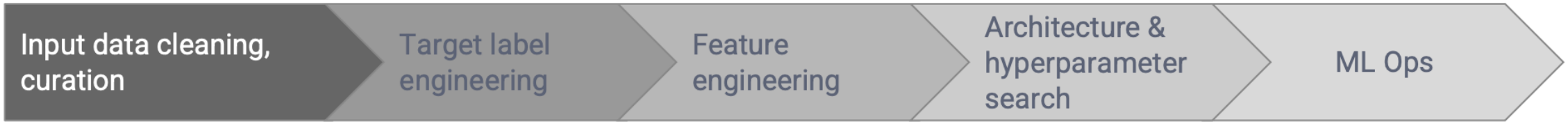


Image credit: Jure Leskovec, http://cs224w.stanford.edu

Standard approach to AI models on relational databases

- Feature selection / engineering using subset of tables / features
- Only a limited set of data is used
- High potential for point-in-time errors / information leakage

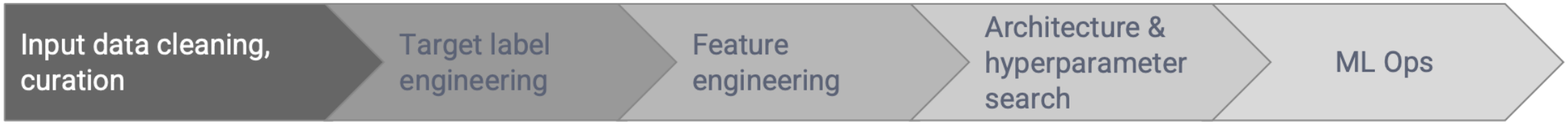


- Typically, several months of development time

Image credit: Jure Leskovec, http://cs224w.stanford.edu

Standard approach to AI models on relational databases

- Feature selection / engineering using subset of tables / features
- Only a limited set of data is used
- High potential for point-in-time errors / information leakage



- Typically, several months of development time
- Can we leverage the data in its existing database form without going through the longest duration tasks?

Image credit: Jure Leskovec, http://cs224w.stanford.edu

Deep learning on databases?

- Can we leverage the data in its existing database form without going through the longest duration model development tasks?

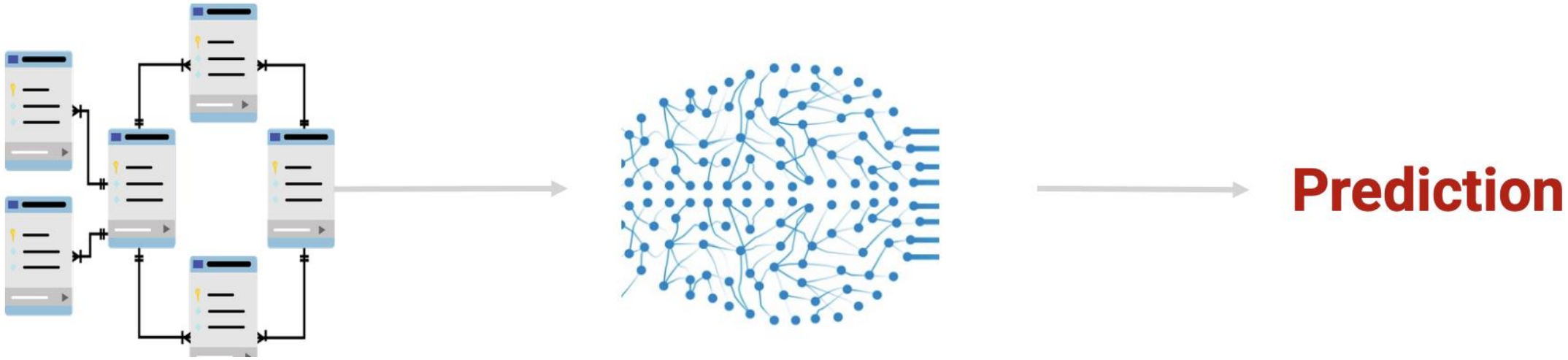


Image credit: Jure Leskovec, http://cs224w.stanford.edu

School of COMPUTING Clemson University

Relational deep learning

Databases are graphs

- Rows in tables can be viewed as nodes
- Relations (foreign keys) between rows in different tables can be viewed as edges between nodes

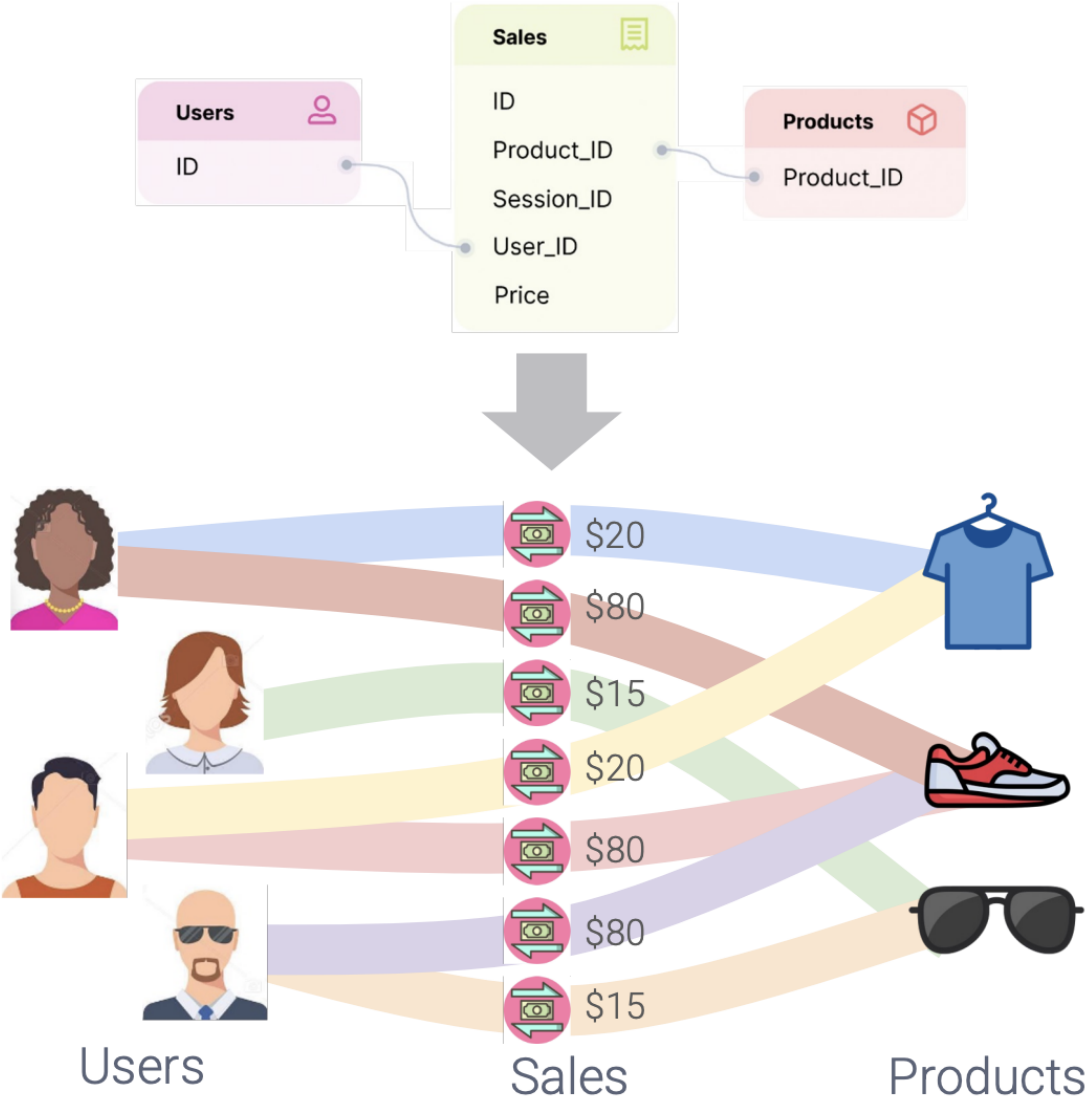


Image credit: Jure Leskovec, http://cs224w.stanford.edu

Databases are graphs

- Rows in tables can be viewed as nodes
- Relations (foreign keys) between rows in different tables can be viewed as edges between nodes
- Can we convert the database to a graph and perform machine learning using GNN concepts?

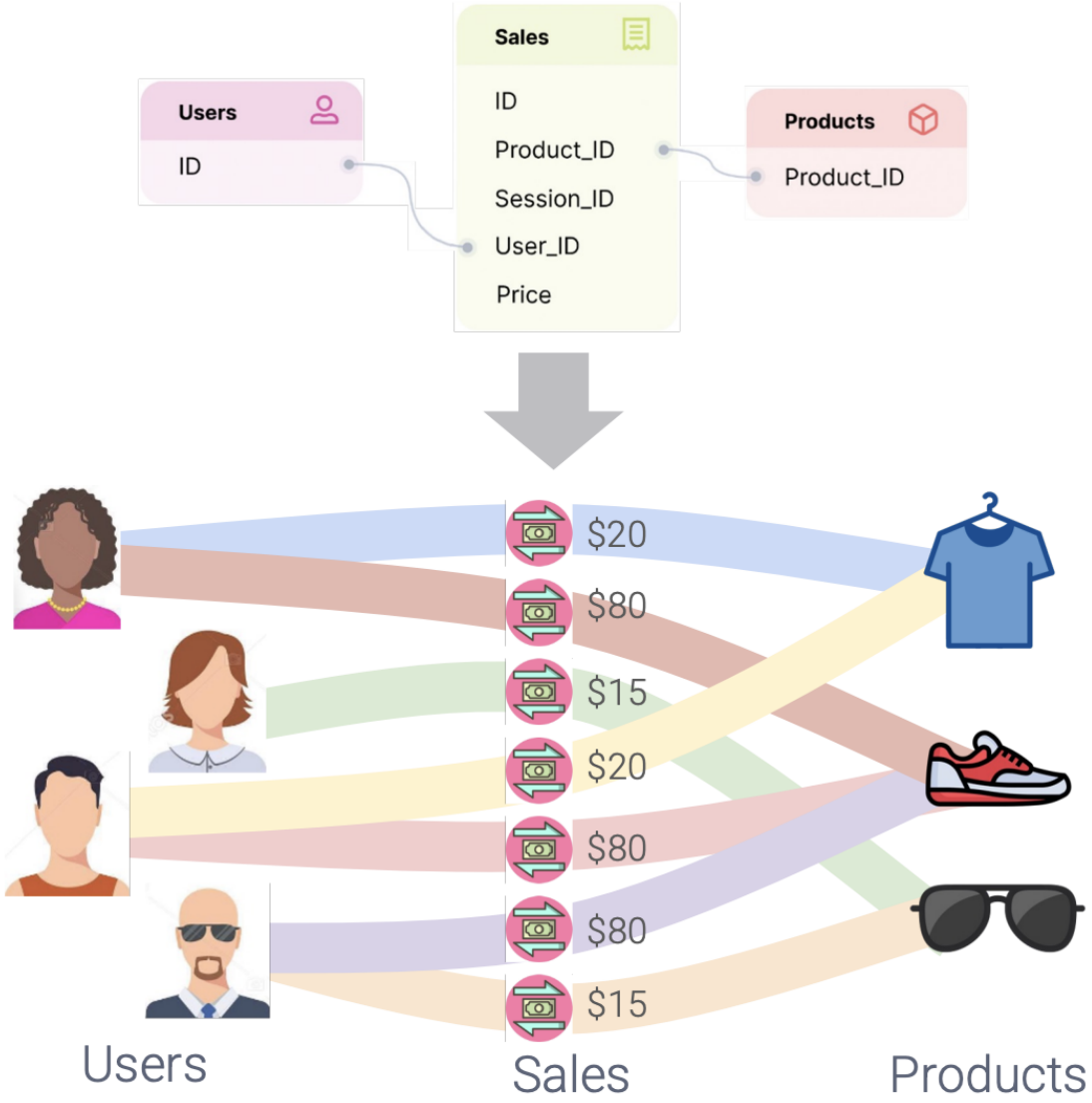


Image credit: Jure Leskovec, http://cs224w.stanford.edu

Database to graph ML pipeline

- Can we convert the database to a graph and perform machine learning using GNN concepts?
- This is the purpose of relational deep learning

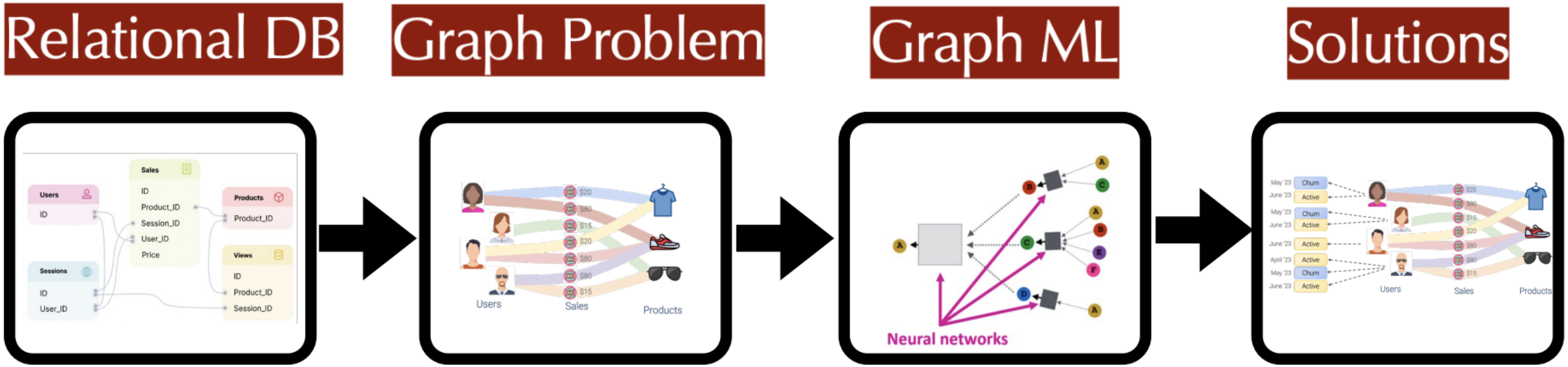


Image credit: Jure Leskovec, http://cs224w.stanford.edu

Relational deep learning pipeline

Three main steps to relational deep learning (RDL)

1. Formulate the predictive task and generate a *training table*
2. Generate the graph representation of the database (including the training table)
3. Develop the graph machine learning model

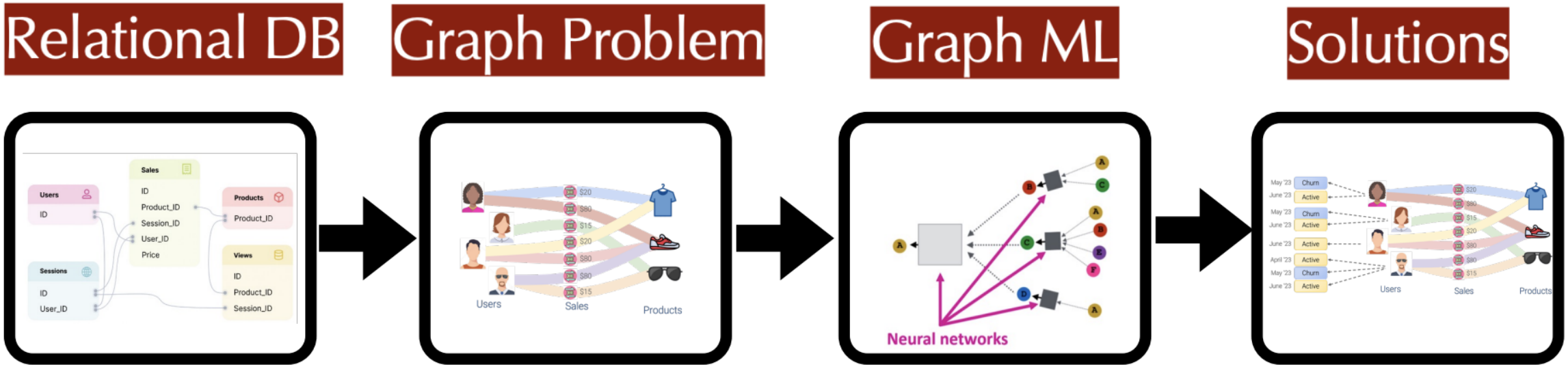


Image credit: Jure Leskovec, http://cs224w.stanford.edu

Step 1: RDL task and training table formulation

Three steps (not to be confused with the 3 RDL steps)

A. Formulate a question for which supervised examples can be created directly from historical data in the database

B. Create a *training table* that contains supervision samples with three features:

   i. label – answer to the question

   ii. entity id (foreign key) to another table (depends on the question)

   iii. seed time (the time point at which the label applies)

C. Integrate the training table into the database (update the schema)

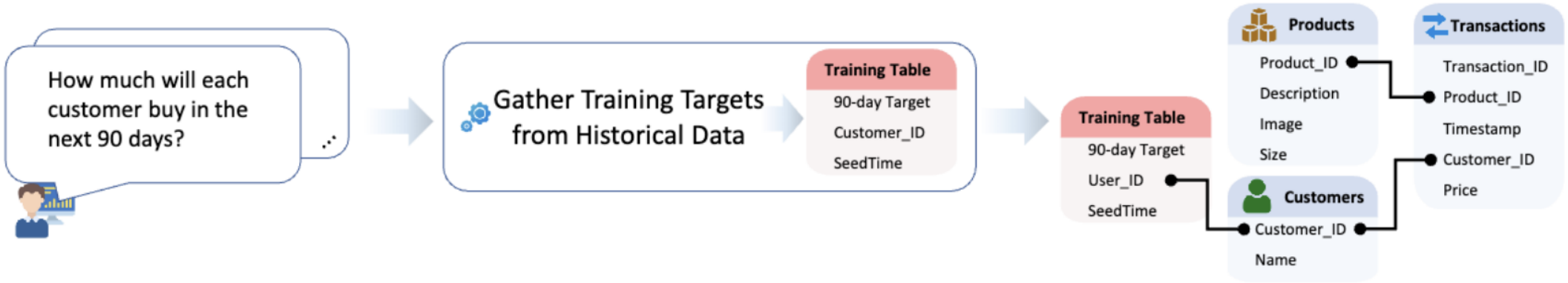


Image credit: Jure Leskovec, http://cs224w.stanford.edu

Step 1: RDL task and training table formulation example

- Predict whether a user is going to churn (not buy anything) in the next 30 days?
  - Samples can be generated from the *Users* and *Sales* tables
  - The entity is a user from the *Users* table
  - The label (churn) is determined by examining the historical entries in the *Sales* table for a given user
- This task is temporal:
  - User's label changes over time
  - Multiple samples can be created for a given user at different times

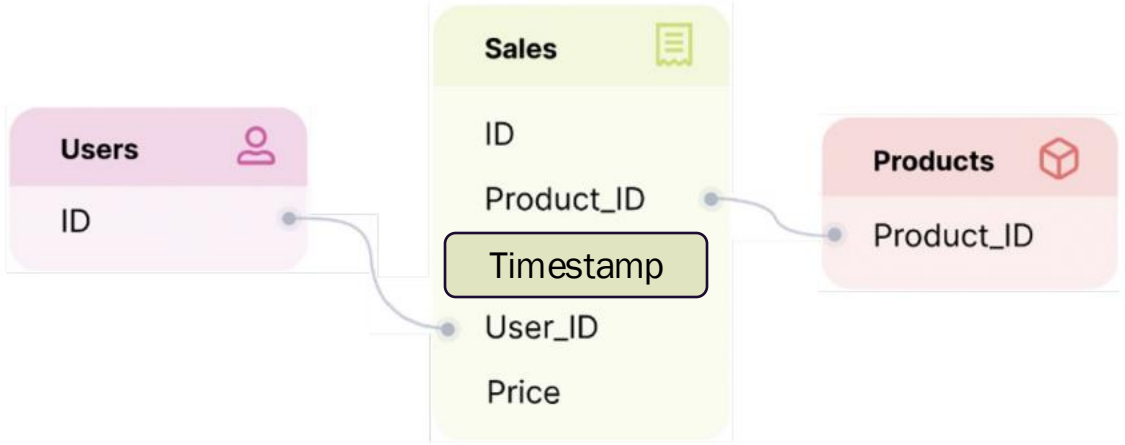
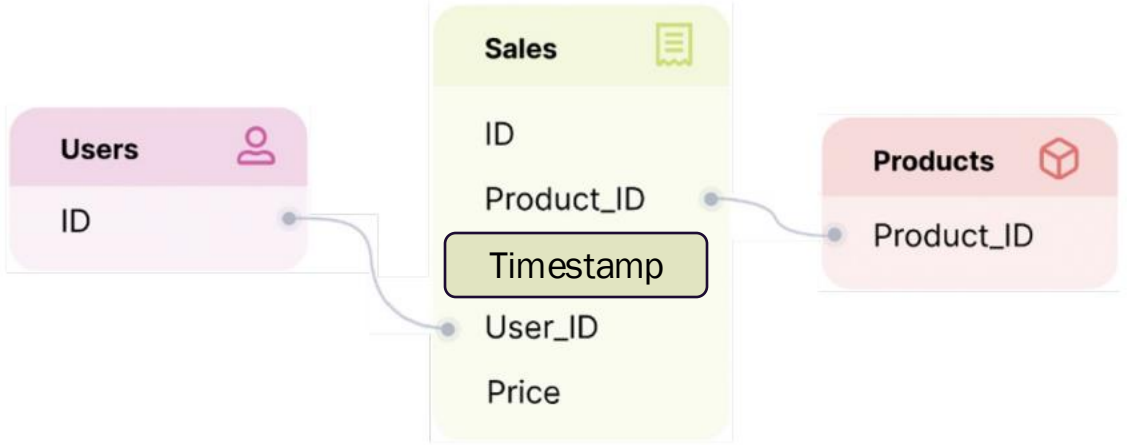


Image credit: Jure Leskovec, http://cs224w.stanford.edu

Step 1: RDL task and training table formulation example

- Predict whether a user is going to churn (not buy anything) in the next 30 days?
- The *training* table contains
  - label – churn (sales in last 30 days)
  - entity id – foreign key to a user_id
  - seed time (the time point at which the label is applies)
- All training table entries are derived directly from the database historical data



Entity ID	Timestamp	Label
99	10172024	1
99	10182024	1
...	...	...
100	10172024	1
100	10182024	0
...	...	...

Image credit: Jure Leskovec, http://cs224w.stanford.edu

Relational deep learning pipeline

Three main steps to relational deep learning (RDL)

1. ✔ Formulate the predictive task and generate a *training table*
2. Generate the graph representation of the database (including the training table)
3. Develop the graph machine learning model

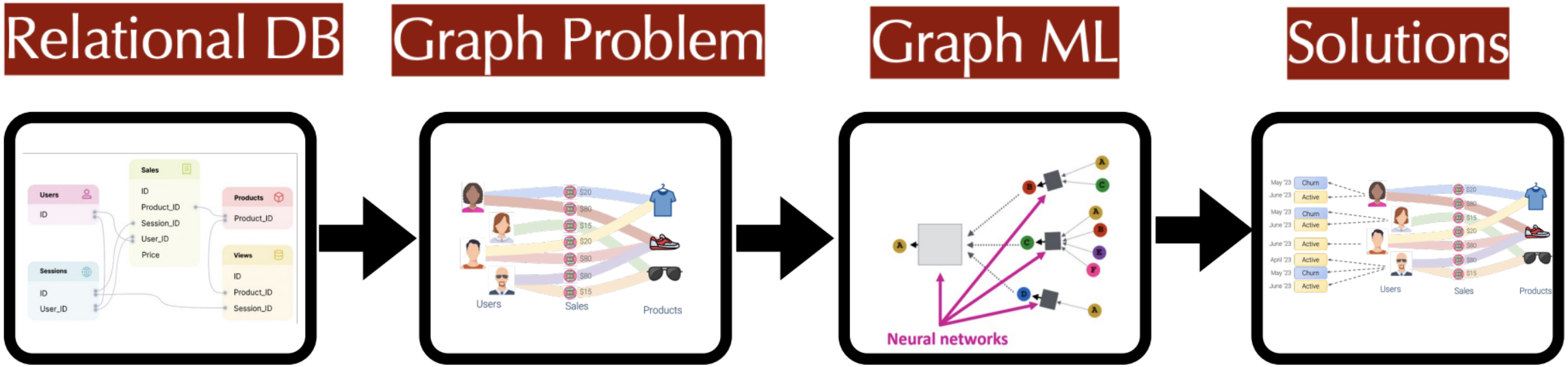


Image credit: Jure Leskovec, http://cs224w.stanford.edu

Step 2: Database graph representation

- Mathematical definition of a database
- The database will be represented by two graphs
  a) Schema graph
  b) Relational entity graph

Step 2: Database mathematical definition

- A database, $(\mathcal{T}, \mathcal{L})$ is a set of tables $\mathcal{T} = \{T_1, \cdots T_n\}$ and links between tables $\mathcal{L} \subseteq \mathcal{T} \times \mathcal{T}$
  - A link $L = (T_{fkey}, T_{pkey})$ exists if there is a foreign key column in the table T_{fkey} that points to the primary key column in table T_{pkey}
  - Each table is a set $T = \{v_1, \cdots, v_{n_T}\}$ whose elements (rows) have four parts:
    1. Primary key p_v
    2. Foreign keys $\mathcal{K}_v \subseteq \{p_{v'} : v' \in T' \; and \; (T, T') \in \mathcal{L}\}$
    3. Attributes (other columns/features)
    4. Timestamp (optional)

Trans ID	User ID	Product ID	Value	Timestamp
...	...	...	...	...
...	...	...	...	...
...	...	...	...	...
...	...	...	...	...
100	51	345	$1000	1710281001
...	...	...	...	...

User Table

User ID	Gender
...	...
51	Male

Product Table

Product ID	Category
...	...
345	Bike

Image credit: Jure Leskovec, http://cs224w.stanford.edu

Step 2: Schema graph

- Given a database, $(\mathcal{T}, \mathcal{L})$, let

  $$\mathcal{L}^{-1} = \{(T_{pkey}, T_{fkey}) \,|\, (T_{fkey}, T_{pkey}) \in \mathcal{L}\}$$

  (mapping from primary keys to foreign keys)
- The schema graph $(\mathcal{T}, \mathcal{R})$ is a homogeneous graph with node set $\mathcal{T}$ and edges $\mathcal{R} = \mathcal{L} \cup \mathcal{L}^{-1}$
  - Nodes are the tables
  - Edges connect tables that point to each other via primary / foreign key associations
  - The schema graph does NOT have the table entities (rows)
- Schema graph represents the table-level structure of the database

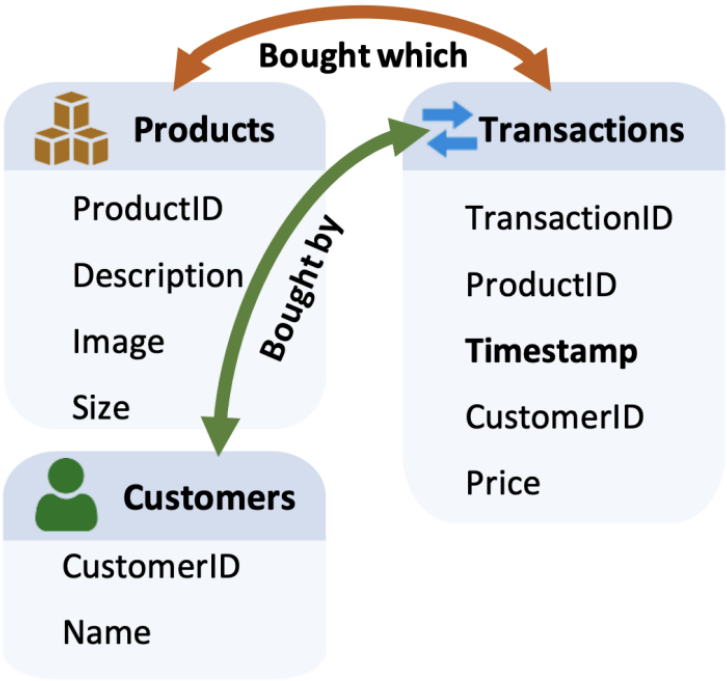


Image credit: Jure Leskovec, http://cs224w.stanford.edu

Step 2: Relational entity graph

- Given a database, $(\mathcal{T}, \mathcal{L})$, the relational entity graph is a heterogeneous graph $G = (\boldsymbol{V}, \boldsymbol{E}, \phi, \psi, \tau)$ where
  - $\boldsymbol{V}$ is the set of nodes corresponding to all rows from all tables in $\mathcal{T}$
  - $\boldsymbol{E}$ is the edge set connecting nodes via primary / foreign keys
  - $\phi(v) \in \mathcal{T}$ is the node type for v
  - $\psi(u, v) \in \mathcal{R}$ is the edge type derived from the schema graph
  - $\tau(v)$ is a mapping to the timestamp of node v
- Nodes (entities) have features derived from the table columns

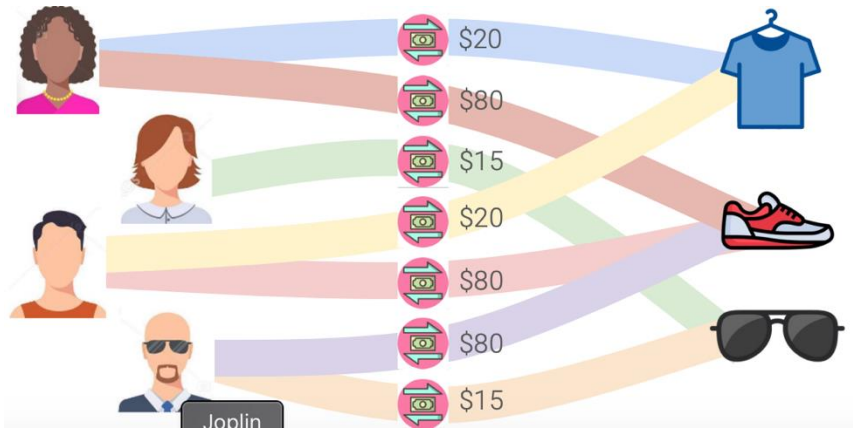


Image credit: Jure Leskovec, http://cs224w.stanford.edu

Step 2: Relational entity graph

- The relational entity graph is completed by:
  - Creating “sample” nodes from the training table rows. The sample nodes have two features: label (from the predictive task), and timestamp
  - Adding edges from the sample nodes to the appropriate entity nodes defined by the foreign key used in the training table

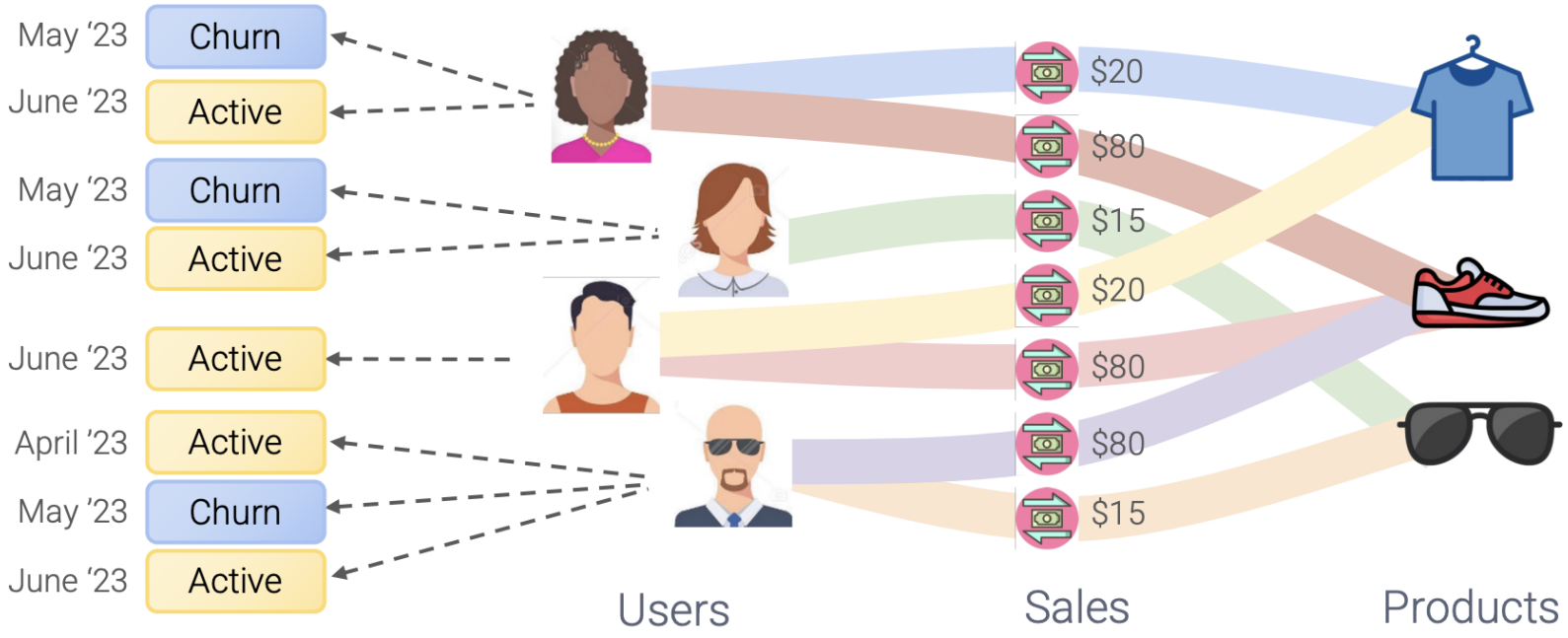


Image credit: Jure Leskovec, http://cs224w.stanford.edu

Relational deep learning pipeline

Three main steps to relational deep learning (RDL)

1. ✔ Formulate the predictive task and generate a *training table*
2. ✔ Generate the graph representation of the database (including the training table)
3. Develop the graph machine learning model

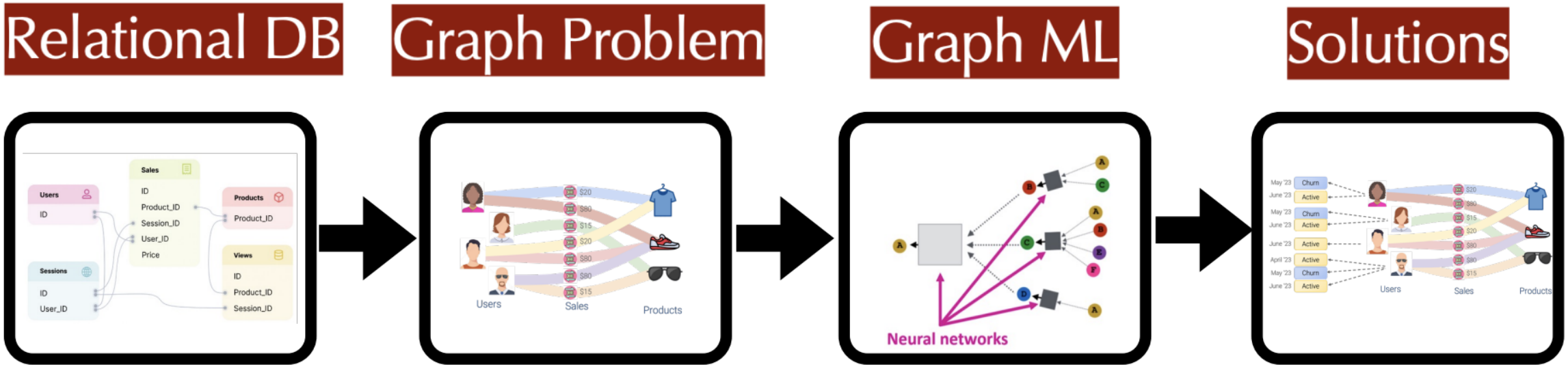


Image credit: Jure Leskovec, http://cs224w.stanford.edu

Step 3: Develop the GNN

- The relation entity graph is a heterogenous graph
- We can apply message passing GNNs to update the node embeddings
- We can create prediction heads for node / graph / edge inference tasks

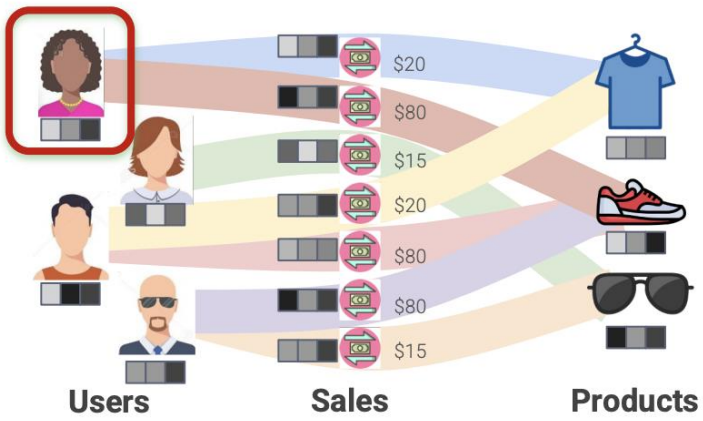
Entity Graph

GNN computation graphs

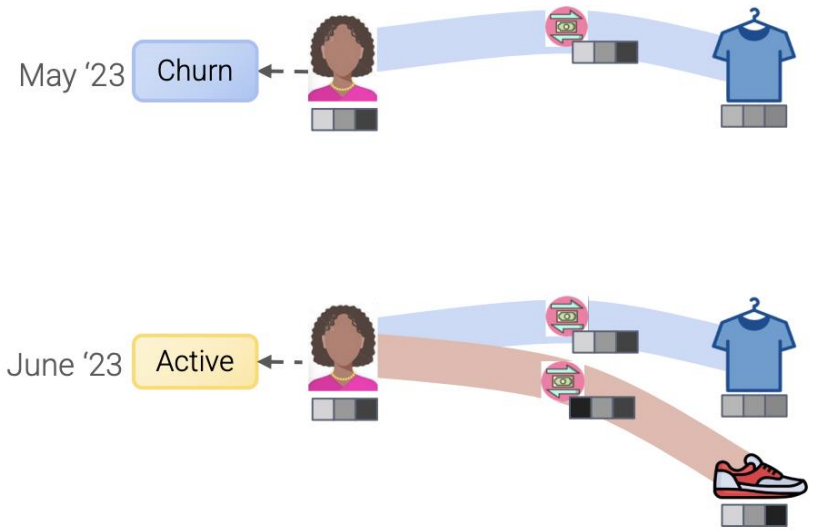
Image credit: Jure Leskovec, http://cs224w.stanford.edu

Step 3: Develop the GNN

- The relation entity graph is a heterogenous graph
- We can apply message passing GNNs to update the node embeddings
- We can create prediction heads for node / graph / edge inference tasks
- There is one issue to resolve – temporality. The entity relation graph has information **for all time** up to present. But training samples correspond to past time points where some of this data did not exist.



Entity Graph

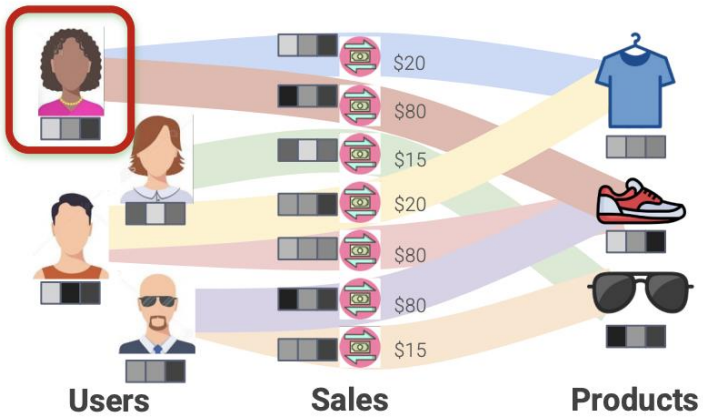


GNN computation graphs

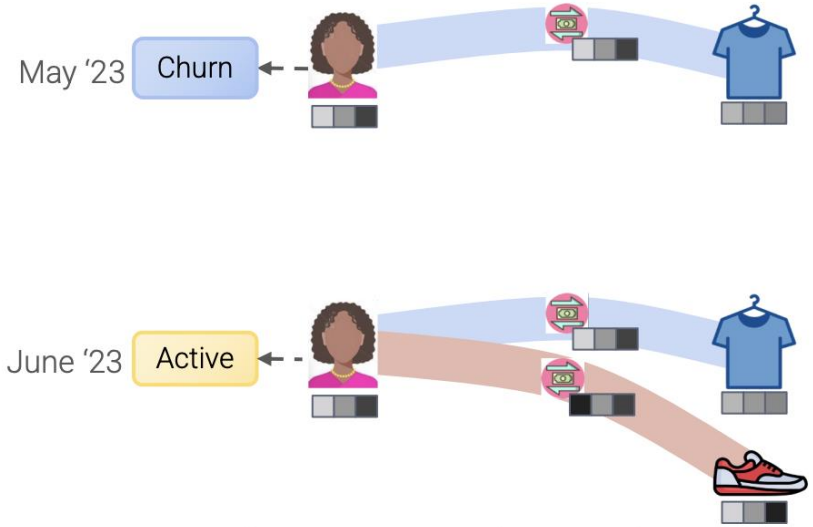
Image credit: Jure Leskovec, http://cs224w.stanford.edu

Step 3: Develop the GNN

- There is one issue to resolve – temporality. We need to prevent “future” data from ”leaking” into the computation for a given sample, i.e., we can only use nodes that represent data that existed at the time of the training sample
- The computation graph for each node is time dependent
- Message aggregation becomes time-dependent
- Sampling over neighbors is time-dependent



Entity Graph



GNN computation graphs

Image credit: Jure Leskovec, http://cs224w.stanford.edu

Step 3: Develop the GNN

- Let's assume we want to use a RGCN to model our GNN
- The standard RGCN node embedding update is given by

$$\mathbf{h}_v^{(k+1)} = \sigma\left(\sum_{r\in R}\sum_{u\in N_r(v)}\frac{1}{c_{v,r}}\mathbf{W}_r^{(k)}\mathbf{h}_u^{(k)} + \mathbf{W}_0^{(k)}\mathbf{h}_v^{(k)}\right)$$

where $N_r(v)$ is the neighborhood of node v for relation type r and $c_{v,r} = |N_r(v)|$

- How can we address the temporality issue?

Step 3: Time consistent computation graphs

- There is one issue to resolve – temporality. Only use nodes that represent data that existed at the time of the training sample
- For a given training sample: Use entity (node) timestamp, $\tau(v)$, information to sample a time consistent subgraph of the relational entity graph
- Similar to GraphSAGE sampling with addition of temporal constraints

Algorithm 1 Time-Consistent Computation Graph

Input: Relational entity graph $G = (\mathcal{V}, \mathcal{E})$, number of hops L, seed node $v_0 \in \mathcal{V}$, seed time $t \in \mathbb{R}$
Input: Neighborhood sizes $(m_1, ..., m_L) \in \mathbb{N}^L$
Output: Computation graph $G_{\text{comp}} = (\mathcal{V}_{\text{comp}}, \mathcal{E}_{\text{comp}})$

$\mathcal{V}_0 \leftarrow \{v_0\}, \quad \mathcal{E}_0 \leftarrow \emptyset$
for $i \in \{1, ..., L\}$ **do**
  for $v \in \mathcal{V}_{i-1}$ **do**
    $\mathcal{E}_i \leftarrow \text{SELECT}_{m_i}(\{(w, v) \in \mathcal{E} \mid \tau(v) \leq t\})$ ▷ Select a maximum of m_i filtered edges
    $\mathcal{V}_i \leftarrow \{w \in \mathcal{V} \mid (w, v) \in \mathcal{E}_i\}$ ▷ Gather nodes for the sampled edges
  end for
end for
$\mathcal{V}_{\text{comp}} \leftarrow \bigcup_{i=1}^{L} \mathcal{V}_i, \quad \mathcal{E}_{\text{comp}} \leftarrow \bigcup_{i=1}^{L} \mathcal{E}_i$

https://arxiv.org/pdf/2312.04615

Step 3: Temporal message passing

- Given a sample from the training table with a seed node v and seed time, t, we can generate a time consistent subgraph using the sampling algorithm in the previous slide
- Now, we perform message passing only on the time-consistent graph

$$\mathbf{h}_v^{(k+1)} = \sigma\left(\sum_{r\in R}\sum_{u\in N_r^{\leq t}(v)} \frac{1}{c_{v,r}} \mathbf{W}_r^{(k)} \mathbf{h}_u^{(k)} + \mathbf{W}_0^{(k)} \mathbf{h}_v^{(k)}\right)$$

where $N_r^{\leq t}(v) = \{u \in \boldsymbol{V} \mid (u,v) \in \boldsymbol{E}, \psi(u,v) = r, \tau(u) \leq t\}$

- $N_r^{\leq t}(v)$ is formed using the sampling algorithm in the previous slide

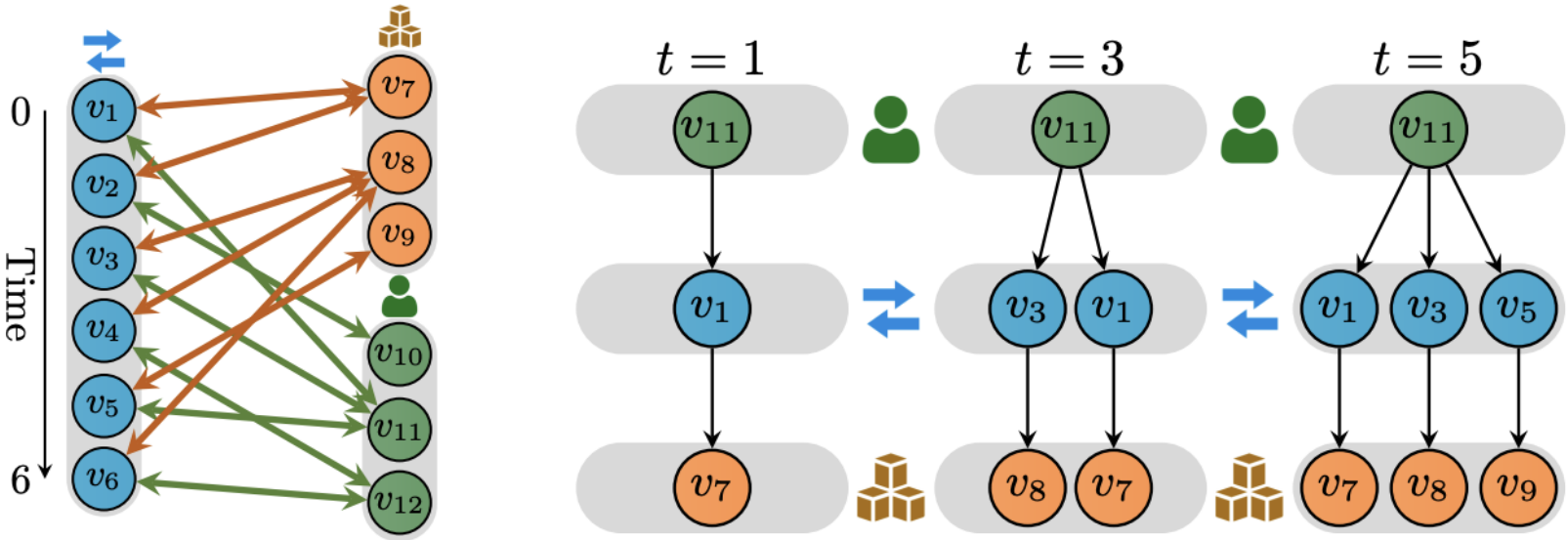
https://arxiv.org/pdf/2312.04615

Step 3: Temporal message passing

- Now, we perform message passing only on the time-consistent graph for each sample

$$\mathbf{h}_v^{(k+1)} = \sigma\left(\sum_{r\in R}\sum_{u\in N_r^{\leq t}(v)} \frac{1}{c_{v,r}}\mathbf{W}_r^{(k)}\mathbf{h}_u^{(k)} + \mathbf{W}_0^{(k)}\mathbf{h}_v^{(k)}\right)$$

where $N_r^{\leq t}(v) = \{u \in \boldsymbol{V} \mid (u,v) \in \boldsymbol{E}, \psi(u,v) = r, \tau(u) \leq t\}$



https://arxiv.org/pdf/2312.04615

Relational deep learning pipeline

Three main steps to relational deep learning (RDL)

1. ✔ Formulate the predictive task and generate a *training table*
2. ✔ Generate the graph representation of the database (including the training table)
3. ✔ Develop the graph machine learning model

What have we gained?

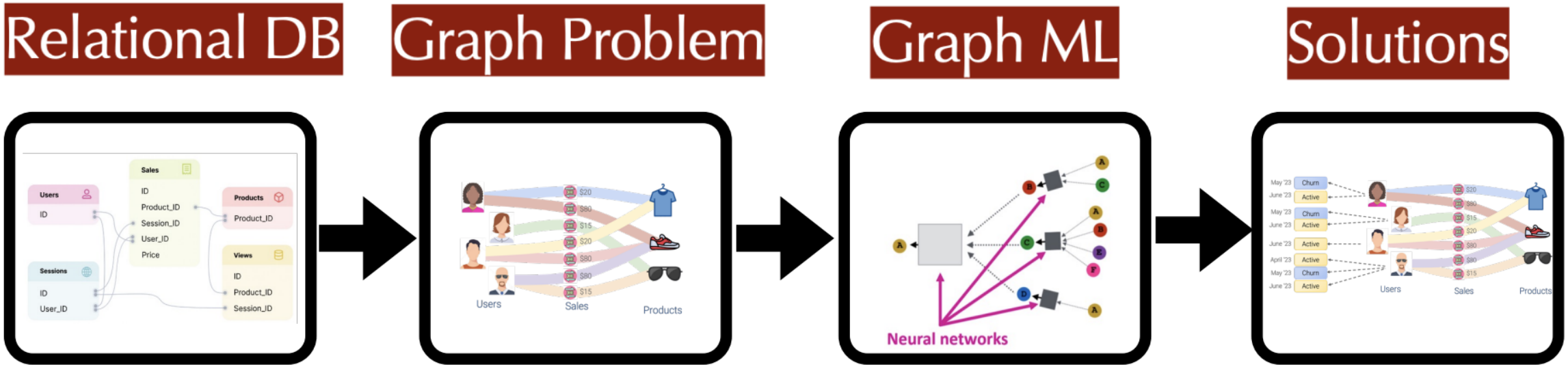
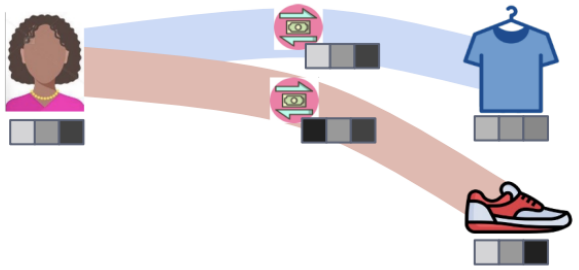


Image credit: Jure Leskovec, http://cs224w.stanford.edu

GNN vs. feature engineering

- GNN learns how to aggregate entity (row) values to form features that are useful for prediction task
- They can discard neighboring node information that is irrelevant for the given downstream task
- They can detect fine-grained patterns within local neighborhoods (e.g., buying behavior over the last year)
- As with deep learning in other domains, we anticipate these will lead to higher performance

GNN-based features:



VS.

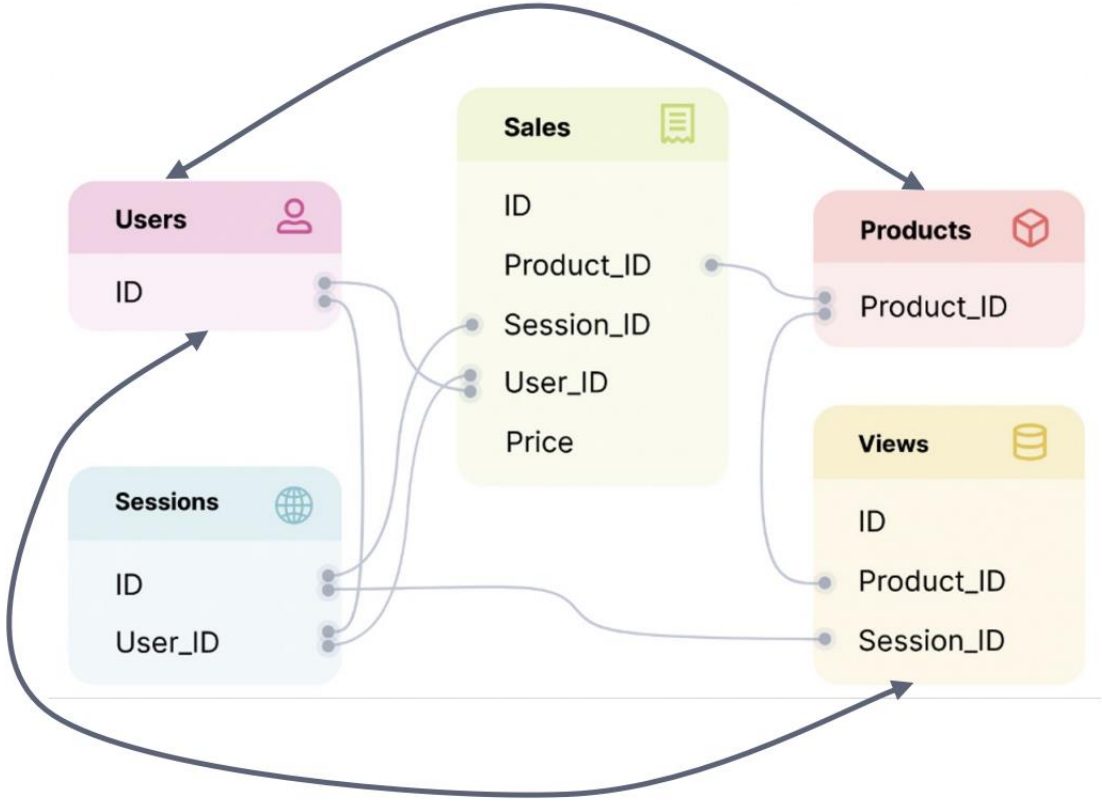
Hand-engineered features:

SUM(TRANSACTIONS.Price) over (-30, 0) days	AVG(TRANSACTIONS.Price) over (-30, 0) days	...

Image credit: Jure Leskovec, http://cs224w.stanford.edu

Information exchange

- GNNs can exchange information across training examples
- Instead of treating examples as isolated, there now exists an inter-dependency between entities
- GNN can use these features to enrich an entity's representation



Information flows between users and products through multiple layers of message passing

Image credit: Jure Leskovec, http://cs224w.stanford.edu

First class temporal information

- Capture fine-grained relative and seasonal features via temporal embeddings
- Avoid data leakage via temporal sampling directly during data loading

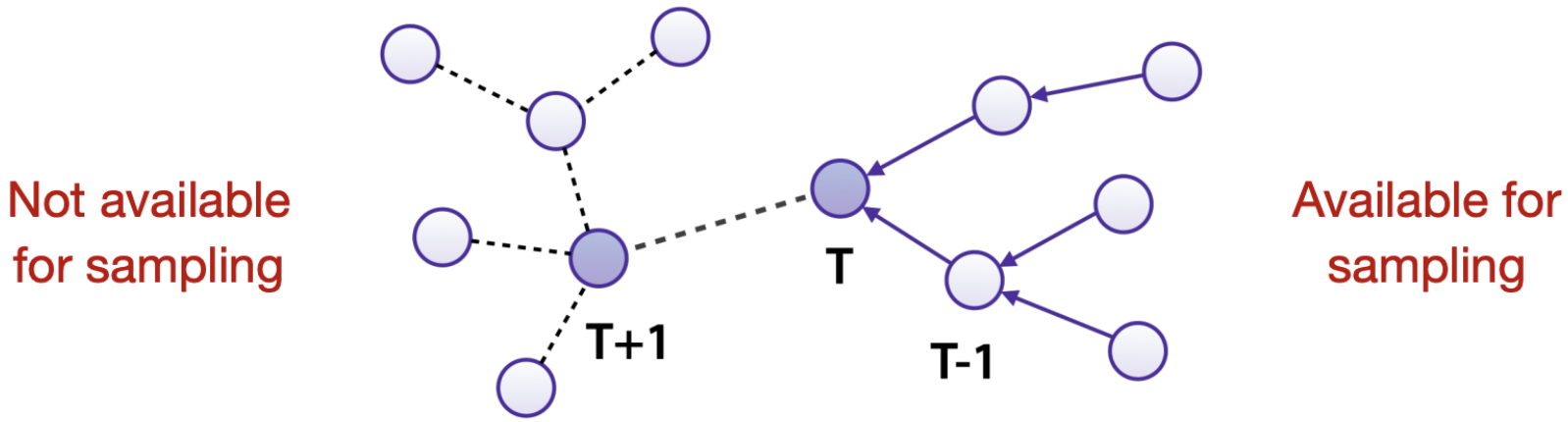


Image credit: Jure Leskovec, http://cs224w.stanford.edu

For more information

- https://relbench.stanford.edu/
- https://arxiv.org/pdf/2312.04615

Relational Deep Learning: Graph Representation Learning on Relational Databases

Matthias Fey[2,*], **Weihua Hu**[2,*], **Kexin Huang**[1,*], **Jan Eric Lenssen**[2,3,*], **Rishabh Ranjan**[1,*], **Joshua Robinson**[1,*], **Rex Ying**[2,4], **Jiaxuan You**[2,5], **Jure Leskovec**[1,2]

[*]Equal contribution. Listed in alphabetical order.

[1]Stanford University
[2]Kumo.AI
[3]Max Planck Institute for Informatics
[4]Yale University
[5]University of Illinois at Urbana-Champaign

RELBENCH: https://relbench.stanford.edu

Image credit: Jure Leskovec, http://cs224w.stanford.edu